

Introduction to Programming

1. Program

Definition:

A **program** is a **set of instructions** written in a programming language that tells a computer **what to do and how to do it**.

Explanation:

- A computer cannot think or decide on its own.
- It works strictly based on instructions given by humans.
- These instructions are written in a structured and logical manner.
- When instructions are combined together to solve a specific problem, it is called a **program**.

Example:

- A program to add two numbers
- A program to calculate student marks
- A program to display "Hello World"

Characteristics of a Program:

- Written using a programming language
- Has a clear objective
- Executes step-by-step
- Produces an output

2. Programming

Definition:

Programming is the process of **designing, writing, testing, debugging, and maintaining programs**.

Explanation:

Programming involves:

1. Understanding the problem
2. Designing a solution (logic or algorithm)
3. Writing code using a programming language
4. Testing the program
5. Fixing errors (debugging)
6. Improving and maintaining the program

Why Programming is Important:

- To automate tasks
- To develop software and applications
- To control hardware
- To solve complex problems

Real-World Examples:

- Mobile apps
- Websites
- Banking systems
- AI and Machine Learning

3. Types of Programming Languages

Programming languages are classified based on **level of abstraction** and **purpose**.

3.1 Low-Level Languages

(a) Machine Language

- Written in binary (0s and 1s)
- Directly understood by the computer
- Very fast execution
- Difficult to write and understand

Example:

10101001 00010101

(b) Assembly Language

- Uses symbols and mnemonics
- Easier than machine language
- Machine-dependent
- Requires an assembler

Example:

ADD A, B

3.2 High-Level Languages

- Easy to read and write
- English-like syntax
- Machine-independent
- Portable

Examples:

- C
- C++
- Java
- Python
- JavaScript

Advantages of High-Level Languages:

- Easy to learn
- Easy to debug
- Faster development
- Less code required

4. Compiler

Definition:

A **compiler** is a program that **translates the entire source code** of a high-level language into machine code **at once**.

How Compiler Works:

1. Reads the complete source code
2. Checks for errors
3. Translates code into machine language
4. Generates an executable file

Characteristics:

- Translation is done once
- Errors are shown after compilation
- Execution is fast

Examples of Compiled Languages:

- C
- C++
- Java (uses compiler + JVM)

5. Interpreter

Definition:

An **interpreter** is a program that **translates and executes code line-by-line**.

How Interpreter Works:

1. Reads one line of code
2. Translates it into machine language
3. Executes it immediately
4. Moves to the next line

Characteristics:

- No separate executable file
- Stops execution when an error occurs

- Slower than compiler

Examples of Interpreted Languages:

- Python
- JavaScript
- PHP

6. Compiler vs Interpreter

Feature	Compiler	Interpreter
Translation	Entire program at once	Line-by-line
Speed	Faster execution	Slower execution
Error Handling	Shows all errors at once	Stops at first error
Output File	Generates executable	No executable file
Examples	C, C++	Python, JavaScript

7. Introduction to Python

What is Python?

Python is a **high-level, interpreted, general-purpose programming language** created by **Guido van Rossum** and released in **1991**.

Why the Name Python?

- Named after the comedy show "**Monty Python's Flying Circus**"
- Not related to the snake

Python Philosophy:

- Simple is better than complex
- Readability counts

8. Features of Python

1. Simple and Easy to Learn

- English-like syntax
- Easy for beginners

2. Interpreted Language

- No compilation needed
- Code runs line-by-line

3. High-Level Language

- No need to manage memory manually

4. Object-Oriented

- Supports classes and objects

5. Portable

- Runs on Windows, Linux, macOS

6. Free and Open Source

- No license cost

7. Large Standard Library

- Built-in modules and functions

8. Dynamically Typed

- No need to declare variable types

9. Applications of Python

1. Web Development

- Django
- Flask

2. Data Science and Analytics

- NumPy
- Pandas
- Matplotlib

3. Artificial Intelligence & Machine Learning

- TensorFlow
- Scikit-learn

4. Automation and Scripting

- Task automation
- File handling

5. Game Development

- Pygame

6. Desktop Applications

- Tkinter
- PyQt

7. Cyber Security

- Ethical hacking tools

8. Education

- Beginner-friendly language

Conclusion

Programming is the foundation of computer science. Python is one of the best languages for beginners due to its simplicity, readability, and wide range of applications. Understanding basic concepts like programs, compilers, interpreters, and programming languages is essential before moving to coding.
